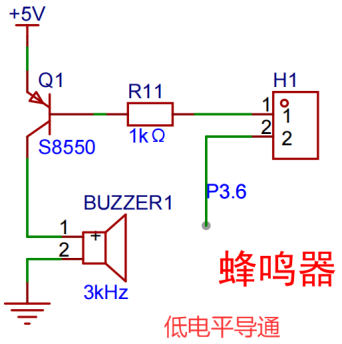
```
主要任务：小车运动
    1、中断
    2、直流电机
    3、小车运动
```

1、延时

```
void delay_ms(int n)    //  使用时提供一个参数 n 代表延时 多少 ms
{
    unsigned char i;
    while(n--)          //  循环执行 n 次,每次为 1ms
    {
        for(i = 0;i < 113;i++); //  执行 113 次 ++ 与比较,时间开销为 1ms
    }
}

将上述代码加入到 main 函数 上面
作用:
    延时若干 ms
    delay_ms(1);        //  延时 1 ms
    delay_ms(500);      //  延时 0.5s
    .....
```

```
#include <reg52.h>      //  头文件,里面存放了很多 声明,帮我们已经声明好的名字

//  一些声明语句
sbit D1 = P1^0;         //  给 P1.0 取了一个名字叫 D1


void delay_ms(int n)    //  使用时提供一个参数 n 代表延时 多少 ms
{
    unsigned char i;
    while(n--)          //  循环执行 n 次,每次为 1ms
    {
        for(i = 0;i < 113;i++); //  执行 113 次 ++ 与比较,时间开销为 1ms
    }
}


//  主函数
void main(void)         //  主函数,程序从主函数开始,也从主函数结束
{
    D1 = 0;             //  P1.0 低电平
    delay_ms(500);      //  延时 500ms
    D1 = 1;             //  P1.1 高电平
    delay_ms(500);      //  延时 500ms
}
```

```
练习:
    蜂鸣器 响一会灭一会(P0.7)
```



2、流水灯

```
C 语言位操作
    把整数转换为二进制,再进行对应的操作
```

(1)按位左移

```
x << n
    将 x 转换为 二进制,整体左移 n 个 bit,左边多的 n bit 不要,右边空出来的 n bit 补0
eg:
    5 << 3
    0000 0101   --->    5
    0010 1000   --->    40

    1 << 2
    0000 0001   --->    1
    0000 0100   --->    4
```

(2)按位与

```
a & b   a,b 为单个 bit,运算结果如下:
a   b   a&b
0   0    0
0   1    0
1   0    0
1   1    1

eg:
    5&3 ---> ???
    0000 0101
&   0000 0011
    0000 0001
```

(3)按位或

```
a | b   a,b 为单个 bit,运算结果如下:
a   b   a|b
0   0    0
0   1    1
1   0    1
1   1    1

eg:
    5|3 ---> ???
    0000 0101
|   0000 0011
    0000 0111
```

(4)按位异或

```
a ^ b   a,b 为单个 bit,运算结果如下:
a   b   a^b
0   0    0
0   1    1
1   0    1
1   1    0

eg:
    5^3 ---> ???
    0000 0101
^   0000 0011
    0000 0110
```

(5)按位取反

```
0 ---> 1
1 ---> 0
eg:
    ~0  --->    1111 1111
    ~3  --->    1111 1100
```

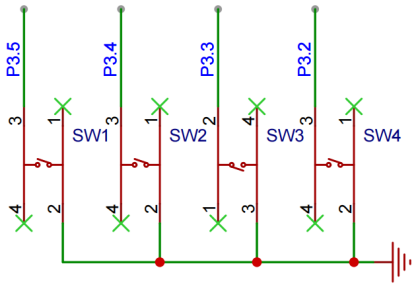
(6)流水灯

```
D1 亮其它灭0.5S     P1 ---> 1111 1110 <--- ~(0000 0001)
D2 亮其它灭0.5S     P1 ---> 1111 1101 <--- ~(0000 0010)
.......
D8 亮其它灭0.5S     P1 ---> 0111 1111 <--- ~(1000 0000)
    P1 = ~(1<<i);

代码实现:
    for(i = 0;i < 8;i++)    //  i 从 0~7
    {
        P1 = ~(1 << i);
        delay_ms(500);      //  延时 500ms
    }
练习:
    D1、D8 亮其它灭0.5S  P1 ---> 0111 1110
    D2、D7 亮其它灭0.5S  P1 ---> 1011 1101
    D3、D6 亮其它灭0.5S  P1 ---> 1101 1011
```

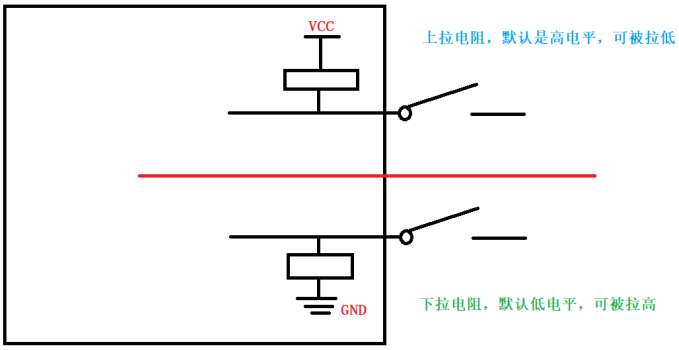
```
    D4、D5 亮其它灭0.5S  P1 ---> 1110 0111
    D5、D4 亮其它灭0.5S  P1 ---> 1110 0111
    ....
    D8、D1 亮其它灭0.5S  P1 ---> 0111 1110

    ~(1 << i)        0 往前走    D1~D8
    ~(1 << (7-i))    0 往后走    D8~D1
    两个结果位与既可!!!
```

3、外部按键

```
Key1    -->     CPU(P3.3,内部有上拉电阻)
    通过分析
    弹起状态     1    -----    高电平
    按下状态     0    _____    低电平
    CPU 可以通过查询(读取) P3.3 这个引脚的电平状态,来知道外部按键 Key1 是否按下
```

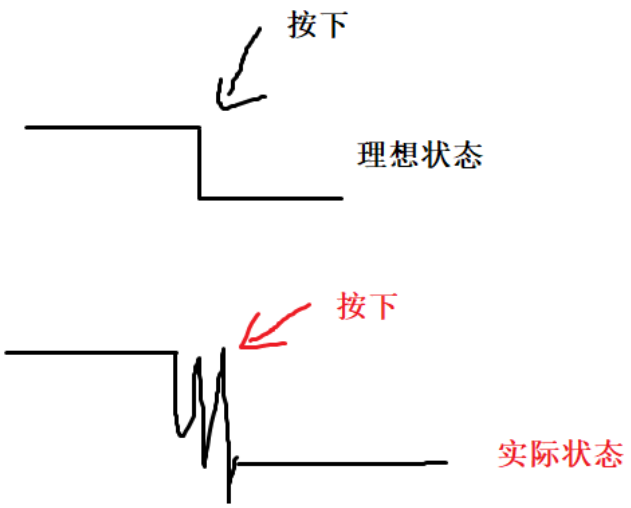


独立按键




```
eg:
    按下 Key1 同时 D1 亮
#include <reg52.h>      //  头文件,里面存放了很多 声明,帮我们已经声明好的名字
sbit D1 = P1^0;         //  给 P1.0 取了一个名字叫 D1
sbit KEY1 = P3^5;       //  给 P3.5 取了一个名字叫 KEY1

void main(void)
{
    if(KEY1 == 1)   //  按键 KEY1 弹起状态,两个等号为判断
    {
        D1 = 1;     //  灯不亮
    }
    else            //  按键 KEY1 按下状态
    {
        D1 = 0;
    }
}
```

练习：
    KEY3 控制 所有 LED 亮灭
    KEY4 控制 蜂鸣器响/不响



"按键抖动"
    产生原因：
        内部结构和外部电路影响。
    一般开关都是采用机械弹性结构，当机械触点断开、闭合，于弹性作用，按下不会立马稳定状态，闭合也不会立马稳定，伴随一定时间抖动。
    抖动时间在 10 ms，刻意快速按下，达到 40~50 ms。为了解决作用的问题，键状态发生改变时，不立即响应，而是先等待稳定之后再进行处理。消抖两种方法：
    (1)硬件消抖　　改变结构/电路(增加成本)
            并联一个电容，利用电容充电放电性质对电压毛刺进行平滑处理
            实际效果不是很好，成本增加，电路复杂，使用不多。
    (2)软件消抖　　延时一段时间
        绝大多数情况都是采用软件消抖。最简单的就是，当按键有变化时，先等待 10 ms 左右延时时间，让抖动消失后再进行按键判断，如果和上一次一样，稳定按键动作。将之前的代码改动一下：

```
unsigned char r1,r2;     // 保存延时消抖前后的状态
r1 = K1;          //  延时前
delay_ms(10);
r2 = K1;          //  延时后
if(r1 == r2)
{
    if(r1 == 0)          //  按键按下
    {
        D1 = 0;          //  点亮 D1
    }
    else                 //  按键弹起
    {

    }
    while(!KEY1);         // 等待按键释放
    delay_ms(10);         // 释放消抖
}
else
{
    //  有抖动
}
```

练习：
    尽量使用消抖的方式，实现上面的练习!!!

```
思考:
    按下 KEY1 实现流水灯
    按下 KEY2 实现蜂鸣器响3s
    测试同时按下会出现什么效果,为什么???
```

4、C语言延时函数的实现

```
晶振频率     Fosc = 11.0592 MHz
晶振周期     t = (1 / 11.0592MHz) s

一个晶振周期(震荡周期)定义为一个节拍，记为 P
2 个节拍定义为一个状态周期 S = 2P

机器周期:
    在芯片内部时间最小单位，其他所有时间都是机器周期的整数倍。
    机器周期为 6 个 S
    T = 6S = 12P = 12 * (1 / 11.0592MHz) s ≈ 1 us(1.085us)

指令周期:
    完成一条指令需要的时间周期
    单指令周期：该指令完成需要一个机器周期
    双指令周期：该指令完成需要两个机器周期
    三指令周期：该指令完成需要三个机器周期
    ......
    1~4 机器周期

    for(i = 0;i < x;i++)     //  汇编指令的条数 8条(8 个机器周期)
    //  循环执行一次 需要 8 条机器指令
            ===>     8 机器周期
    写一个循环，延时 1ms
        会执行 x 次
        8 * T * x = 1 ms
        x = 1 / 8 / T = 1000 / 8.8 ≈ 113

    在晶振频率为 11.0592 MHz 的 C51 单片机，如下 C 代码，需要 1ms 延时
        for(i = 0;i < 113;i++)
毫秒级延时如下:
    void delay_ms(int n)
    {
        unsigned char i;
        while(n--)
        {
            for(i = 0;i < 113;i++); //  每做 113 次代表 1ms
        }
    }
```

5、中断

(1)轮询

```
分析以下代码:
    while(1)
    {
        if(KEY1 == 0)
        {
```

```
            D1 = 0;
            delay_ms(4000);
            D1 = 1;
        }
        if(KEY2 == 0)
        {
            D2 = 0;
            delay_ms(4000);
            D2 = 1;
        }
    }

试验发现:
    同一时间段按下 KEY1 和 KEY2 只能显示一个灯
    以上代码,不停的查询 KEY1 和 KEY2 引脚电平状态,来判断按键是否按下,这种方式称之为 "轮询"
轮流询问,存在天生缺陷:
    ①浪费 CPU
    ②轮询由时间差,不实时
    ......
有人提出来,能不能不用轮询,等有事件发生时,直接通知 CPU
    这个通知 ---> 中断
```

(2)中断

```
一般定义为打断 CPU 指令执行顺序的事件。
中断源:
    产生中断的设备
C51 支持 5 个中断
    外部中断 0
    定时器 0
    外部中断 1
    定时器 1
    串口中断
C51 中断处理机制
    当 main 函数执行的任意时刻,来了一个中断,此时 CPU 立马中止 main 执行,切换到 中断模式,然后
到指定的位置去执行中断处理函数;最后再回到断点出继续执行后续 main
指定的位置???
        编号 入口地址 中断源
        0    0003H    外部中断 0
        1    000BH    定时器 0
        2    0013H    外部中断 1
        3    001BH    定时器 1
        4    0023H    串口中断
中断处理函数?
    响应中断,C 语言如何实现用户的中断处理函数,如下格式:
    void 中断处理函数名() interrupt 编号
    {
        需要怎么处理
    }
    中断处理函数名:
        随便来,合法且不与关键字相同即可
    interrupt:
        关键字代表该函数是一个中断处理函数
    编号:
        中断编号,见上
    中断处理函数 不需要被主动调用,只要产生某个中断事件,CPU 会主动执行
```

(3)外部中断

```
以外部中断为例,如何操作
    外部中断0 P3.2
    外部中断1 P3.3 ---> KEY1
①外部中断触发方式由 IT0/IT1 分别控制外部中断0,1 的触发方式
    ITx = 0 低电平触发
    ITx = 1 下降沿触发
②外部中断的开关(使能)
    由 EX0/EX1 分别控制外部中断0,1 的开关
    EX0 = 1 ---> 使能外部中断0
    EX1 = 1 ---> 使能外部中断1
    ET0 = 1 ---> 使能定时器中断0
    ET1 = 1 ---> 使能定时器中断1
    ES = 1 ---> 使能串口中断
③总中断开关(使能)
    EA 控制所有中断的使能
    EA = 1
eg:
    // 配置外部中断的函数,以 外部中断1 为例
    void int1_config()
    {
        IT1 = 1;    // 外部中断触发方式 下降沿触发
        EX1 = 1;    // 使能外部中断1
        EA = 1;     // 使能总中断
    }
    void int1_isr() interrupt 2
    {
        // 按需求实现
    }
    void main()
    {
        int1_config(); // 配置好
        while(1)
        {
        }
    }
练习:
    使用 外部中断 实现按键检测
    第一次按下 KEY3(外部中断1) 点亮 D1
    第二次按下 KEY3(外部中断1) 熄灭 D1
    ...
    重复实现
    需要使用消抖把上面的代码优化
```

6、电机

```
常见电机有以下三种:
    a.步进电机
        一种将电脉冲信号转换为对应角位移/线位移 的电动机.
        一次脉冲信号,电机内部的转子会转动一个角度/一步.

    b.舵机
        可以精确控制角位移和线位移,但是对于角度有限制.
        有一定的力矩位置在某个角度,常用于机器人的关节部位.
```

```
    c.直流电机            <---
        接通直流电就可以转动,可以控制转动速度(电流越大,转的越快);生活中非常常见,俗称马达.
        原理: 通电就能转动,反向通带你就会反转;电势差越大,转的越快

每个直流电机都有两个触点,假设为 a b
    如果 a = 1,b = 0; 电机正转
    那么 a = 0,b = 1; 电机反转
    a = 0,b = 0;a = 1,b = 1;     电机不转
两个电机,一共有四个触点,记为 left_a,left_b,right_a,right_b;
    left_a = 1
    left_b = 0       左轮正转
    right_a = 1
    right_b = 0      右轮正转
小车的运动状态即为前进,其它运动同理
```

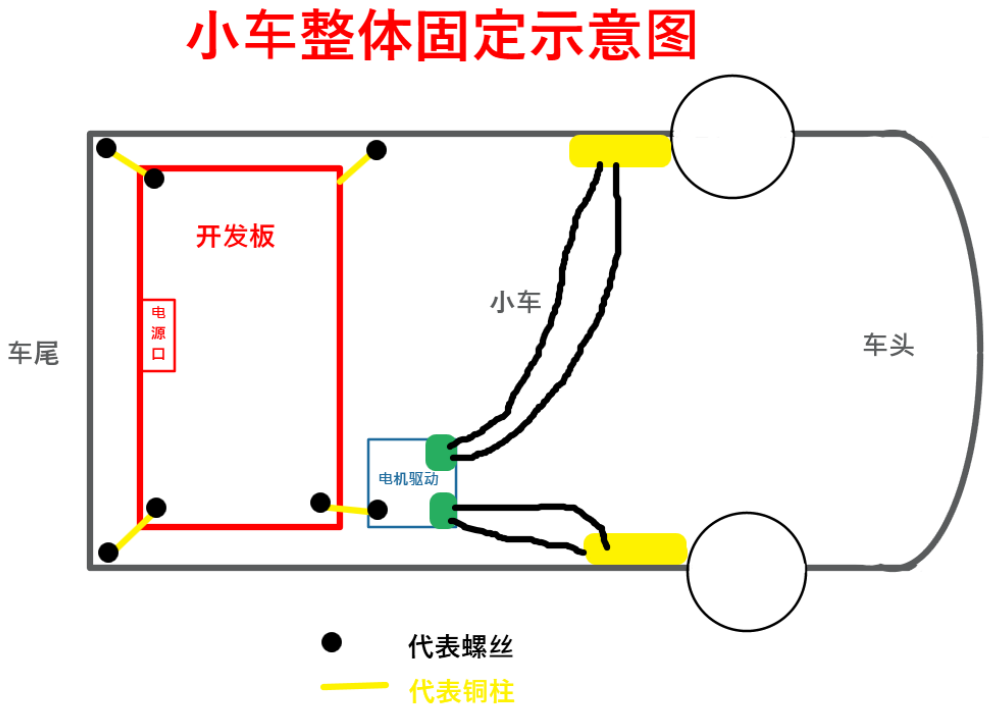
7、控制原理

```
两个电机通过杜邦线与 YL-86 电机驱动模块连接
该模块的硬件参数参考 资料中 《YL-86使用说明.pdf》
    4P 绿端子分别接在 两个直流电机之上
    6P 黑色排针,暂未连接,需要连接在开发板上
        B-1A        模块的输入信号,对应左轮的 a;接在开发板的某个 GPIO 口
        B-2A        模块的输入信号,对应左轮的 b;接在开发板的某个 GPIO 口
        GND         接在开发板的 GND 引脚
        VCC         接在开发板的 5V/3V3 引脚
        A-1A        模块的输入信号,对应右轮的 a;接在开发板的某个 GPIO 口
        A-1B        模块的输入信号,对应右轮的 b;接在开发板的某个 GPIO 口
只需要使用 6 根杜邦线连接这6个引脚到板子上既可
快速测试电机正反转:
    只需要三根杜邦线既可;
        一根 开发板 VCC 接 电机驱动 VCC,
        一根 开发板 GND 接 电机驱动 GND,
        一根 一端接在板子的 GND,一端随便接 电机驱动的剩余四个引脚的一个上电观察运动情况;
    eg:
        ①B-1A 接在 开发板的 GND 上,小车右轮正转;
        ②A-1B 接在 开发板的 GND 上,小车左轮反转;
        A-1B <--> left_a
        A-1A <--> left_b
        B-1A <--> right_b
        B-2A <--> right_a
```

8、小车运动

```
按照上述测试方法,进行代码编写,建议连接在 P1 组上:
sbit left_a = P1^0;     //  P1^0 使用杜邦线接在电机驱动模块的 A-1B
sbit left_b = P1^1;     //  P1^1 使用杜邦线接在电机驱动模块的 A-1A
sbit right_a = P1^2;    //  P1^2 使用杜邦线接在电机驱动模块的 B-2A
sbit right_b = P1^3;    //  P1^3 使用杜邦线接在电机驱动模块的 B-1A

void go_up(void)        //  小车前进
{
    left_a = 1;         //  左轮正转
    left_b = 0;
    right_a = 1;        //  右轮正转
    right_b = 0;
```

```
}

void go_back(void)       //  后退
{
    left_a = 0;          //  左轮反转
    left_b = 1;
    right_a = 0;
    right_b = 1;         //  右轮反转
}

//  直角左转弯
void go_left(void)
{
    left_a = 1;          //  左轮不动
    left_b = 1;
    right_a = 1;         //  右轮正转
    right_b = 0;
}
//  其它运动状态同理 .......

练习:
    使用程序控制小车的各种运动情况
注意:
    接上电机驱动模块可能导致烧录电压不够,先拔掉电机驱动模块的VCC,烧录后再接上既可
```



9、占空比

```
完成小车走直线,可能有个轮子快一点,有个轮子慢一点
    那么就需要把快的轮子变慢,或者把慢的轮子变快
解决:
    ①慢的轮子变快,需要增大电势差;不好实现
    ②快的轮子变慢,需要减小电势差;如何减小???
        使用等效电压, 规定一个较小小的时间为一个周期
        高电平时间占周期的时间的比值称之为 占空比, 可以减小电压
        具体操作:
```

```
            假设以 10 ms 为一个周期，高电平持续时间为 8ms
            那么 V = V总 * (8/10) = 0.8V总，降低电势差
    如左轮快,则可以将前进修改为:
        void go_up_v2(void) //  总周期为 10ms
        {
            right_a = 1;    //  右轮一直全速前进
            right_b = 0;
            left_a = 1;     //  左轮差速前进
            left_b = 0;
            delay_ms(8);    //  高电平 8 ms
            left_a = 0;     //  左轮停止
            delay_ms(2);    //  低电平 2 ms
        }
        占空比为 80%，具体的比值，由小车决定。
```

```c
#include <reg52.h>      //  头文件,里面存放了很多 声明,帮我们已经声明好的名字
sbit left_a = P1^0;     //  P1^0 接在 A-1A
sbit left_b = P1^1;     //  P1^1 接在 A-1B
sbit right_a = P1^2;    //  P1^2 接在 B-2A
sbit right_b = P1^3;    //  P1^3 接在 B-1A
void delay_ms(int n)
{
    unsigned char i;
    while(n--)
    {
        for(i = 0;i < 113;i++); //  每做 113 次代表 1ms
    }
}
//  小车运动方式的函数
void go_up(void)        //  小车前进
{
    left_a = 1;         //  左轮正转
    left_b = 0;
    right_a = 1;        //  右轮正转
    right_b = 0;
}
void go_back(void)          //  后退
{
    left_a = 0;         //  左轮反转
    left_b = 1;
    right_a = 0;
    right_b = 1;        //  右轮反转
}
//  直角左转弯
void go_left(void)
{
    left_a = 1;         //  左轮不动
    left_b = 1;
    right_a = 1;        //  右轮正转
    right_b = 0;
}
//  其它同理
void go_up_v2(void) //  总周期为 10ms,占空比为 80%，具体的比值，由小车决定。
{
    right_a = 1;    //  右轮一直全速前进
    right_b = 0;
    left_a = 1;
    left_b = 0;
    delay_ms(8);    //  高电平 8 ms
    left_a = 0;
    delay_ms(2);    //  低电平 2 ms
}a
void main(void)
{
    while(1)
    {
        go_up_v2();     //  前进
    }
}
```